

Agentic Supervision Harness

Proposed Architecture and Implementation Plan

LIBERO proof of concept -> Neotix towel and T-shirt folding

1. Design Basis and Scope

The supervisor receives live visual observations, the action chunk proposed by the VLA, and a growing memory of previous corrections; it may pass the action through, apply a small end-effector adjustment, choose among sampled candidate chunks, or request a short re-plan [1]. The proposal explicitly keeps the policy frozen and identifies latency, safe overrides, cross-episode memory, and simulation in LIBERO/LIBERO-Plus as core requirements.

Two client-provided papers provide useful architectural ideas without being copied directly as follows:

- ENPIRE organizes robot improvement as a repeatable loop of environment interaction, rollout, verification, and refinement, with explicit safety and automatic verification [2].
- ASPIRE emphasizes fine-grained multimodal execution traces, failure attribution, validated repairs, and a reusable skill library [3].

Our harness should adapt these principles to achieve inference-time supervision of an already-trained policy rather than code generation or policy retraining.

2. Architectural Principles

- Frozen-policy first: the base policy is treated as an immutable black box during the proof of concept.
- Intervene only when necessary: the harness should preserve the base policy when it is behaving correctly and avoid over-correction.
- Constrained correction space: the supervisor chooses from a small, machine-checkable set of interventions rather than emitting arbitrary low-level robot commands.
- Trace before sophistication: high-quality logs and outcome verification are more important initially than a complex multi-agent framework.
- Validated memory only: a correction enters cross-episode memory only after evidence shows that it improved or recovered the rollout.
- Simulation-first interface stability: component APIs used in LIBERO should be designed so that the environment adapter can later be replaced by the Neotix robot API.
- Latency is measured from day one but optimized after correctness is demonstrated.

3. Proposed System Architecture

The harness is best implemented as a modular Python state machine around the policy rather than as a monolithic LLM prompt. A single orchestrator coordinates policy inference, failure gating, supervisor inference, correction generation, safety validation, execution, logging, and memory updates. This makes each component independently testable and allows the team to replace the VLM, memory backend, or environment without changing the whole system.

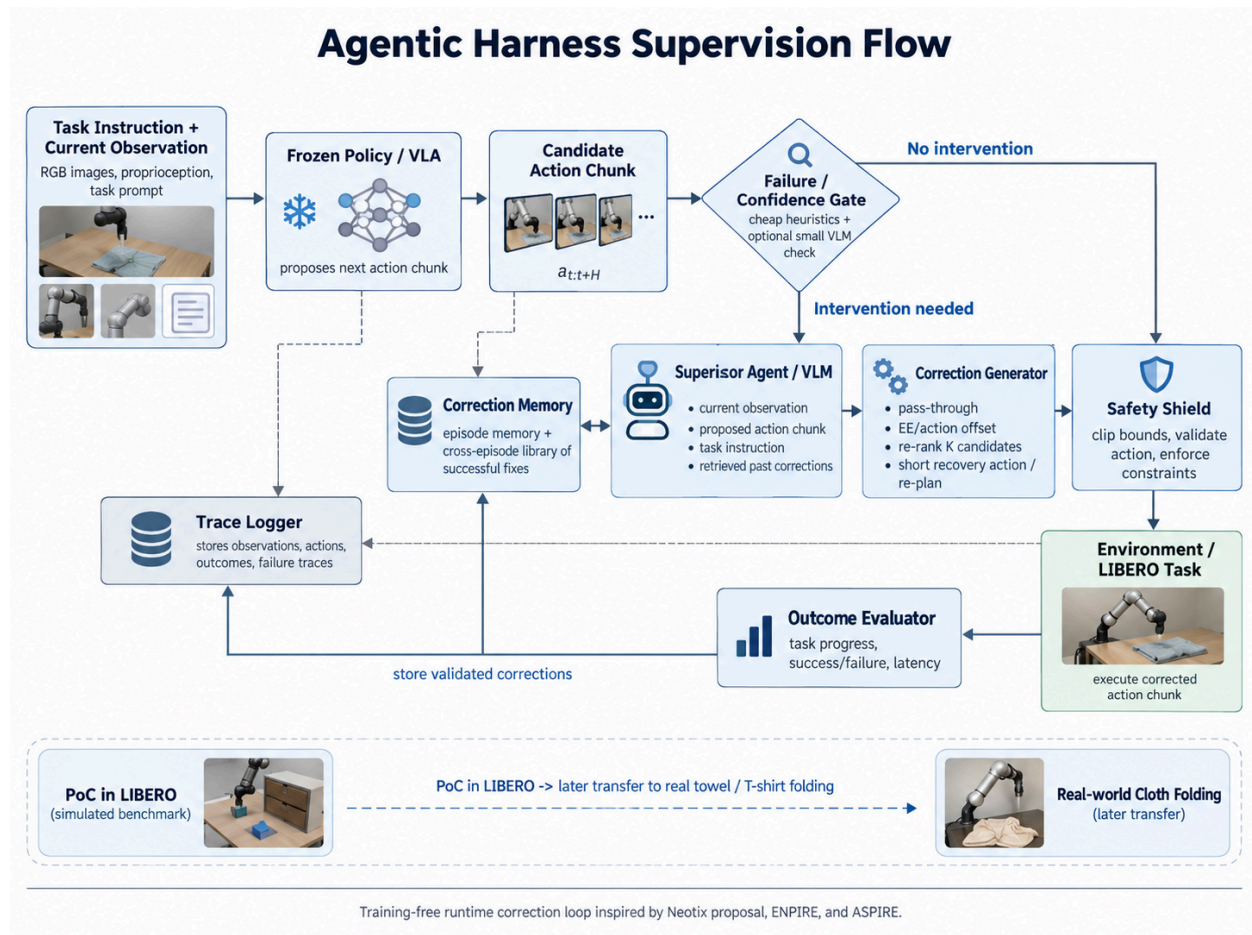


Figure 1. Proposed agentic supervision flow. The LIBERO proof of concept uses the same logical interfaces intended for later transfer to the physical folding setup.

3.1 Component Responsibilities

Component	Inputs	Outputs	Primary responsibility
Environment Adapter	Task instruction, action	RGB / proprioception / reward / done	Normalizes LIBERO now; later wraps Neotix robot interface.
Frozen Policy Adapter	Observation + instruction	Action or action chunk	Calls the selected Hugging Face policy/VLA without updating weights.
Failure / Confidence Gate	Observation, recent progress, proposed action	intervene: bool + reason + confidence	Cheap trigger that prevents calling the supervisor agent on every control step.
Supervisor Agent	Observation, proposed action, trace summary, retrieved corrections	Structured intervention decision	Diagnoses likely failure and selects one bounded recovery option.
Correction Generator	Supervisor decision + base action	Corrected action/chunk	Maps symbolic correction into executable action-space values.
Safety Shield	Candidate corrected action	Validated / clipped action or reject	Enforces action bounds, workspace constraints, and maximum correction magnitude.
Trace Logger	All state transitions and timestamps	Episode trace	Provides evidence for debugging, evaluation, and later failure attribution.
Correction Memory	Validated traces + query context	Top-k relevant corrections	Reuses successful failure-recovery patterns across episodes.
Outcome Evaluator	Post-action observation, reward, done, trace	progress / success / failure labels	Determines whether an intervention helped and whether it should be stored.

4. Runtime Supervision Process

1. Reset the LIBERO environment and obtain the task instruction and initial observation.
2. Pass the current observation to the frozen policy and obtain the next action (or action chunk).
3. Run the failure/confidence gate using the observation, recent trajectory, task progress, and proposed action.
4. If risk is low, send the base action through the safety shield and execute it unchanged.
5. If risk is high, retrieve a small number of relevant validated corrections from memory and call the supervisor VLM.

6. Require the VLM to return a strict structured decision: PASS, OFFSET, SELECT_CANDIDATE, RECOVERY, or TERMINATE, plus bounded parameters and a short reason.
7. Convert the decision into an executable correction, validate it through the safety shield, and execute it.
8. Record the before/after observations, policy proposal, supervisor decision, timing, reward/progress, and episode outcome.
9. When the outcome evaluator confirms that a correction was useful, store a compact correction record for future retrieval.

4.1 Recommended Supervisor Action Set for the PoC

Decision	Meaning	Why useful in PoC	Constraint
PASS	Execute the policy action unchanged.	Prevents the supervisor from becoming the controller.	Default when confidence is high.
OFFSET	Add a small delta to the action / end-effector target.	Closest to the Neotix example of reaching slightly farther before grasping.	Clip per-dimension magnitude; never bypass safety.
SELECT_CANDIDATE	Choose one action from K policy samples.	Lets the supervisor improve behavior without synthesizing a new low-level trajectory.	Only if the policy can cheaply provide K candidates.
RECOVERY	Execute a short predefined recovery primitive.	Useful for repeatable failures such as back-off, re-approach, open gripper, or retry.	Recovery primitives are human-defined and task-safe.
TERMINATE	Stop a clearly unrecoverable or unsafe episode.	Prevents wasting the episode budget and simplifies error analysis.	Used conservatively.

For the first working prototype, we will implement PASS + OFFSET only and then later add candidate re-ranking and recovery primitives after the end-to-end loop is stable. This keeps the causal question clear: can a VLM recognize a bad action and apply a small correction that improves success?

5. Failure Detection and Intervention Trigger

Calling a VLM at every simulator step is unnecessary and will dominate latency. We can use a two-stage trigger:

- Stage A - cheap gate every step: heuristics based on task progress, repeated actions, gripper state, object-to-target distance (when available from simulator state for evaluation), action magnitude, and lack of state change over a short window.
- Stage B - VLM diagnosis only on suspicious states: feed one or a few keyframes plus a compact trace and ask whether the proposed action is consistent with the instruction and current scene.

However, we should be careful not to give the supervisor privileged simulator state that will not exist on the real robot unless it is explicitly part of an ablation. The production path should rely on RGB observations, proprioception that the deployed controller will have, task text, and the proposed action. LIBERO state variables can still be used by the evaluator to score progress and diagnose experiments.

6. Supervisor Prompt and Structured Output

Example Schema:

```
SupervisorDecision = {  
  "intervene": true | false,  
  "failure_type": "misalignment | premature_grasp | stalled | wrong_direction | other",  
  "decision": "PASS | OFFSET | SELECT_CANDIDATE | RECOVERY | TERMINATE",  
  "offset": [dx, dy, dz, droll, dpitch, dyaw, dgrip] | null,  
  "candidate_id": int | null,  
  "confidence": 0.0-1.0,  
  "reason": "one short sentence"  
}
```

We can use Pydantic models to validate this object. The supervisor prompt should contain the task instruction, selected keyframes, a compact summary of the recent trajectory, the policy proposal in a human-readable representation, the allowed intervention menu, explicit numeric bounds, and at most a few retrieved correction examples.

7. Cross-Episode Correction Memory

ASPIRE shows the value of storing validated repairs as compact reusable guidance rather than retaining entire conversations or raw trajectories [3]. For this project, we can use the same principle but store intervention records rather than code skills. A memory record should be small enough to retrieve quickly and rich enough to explain when a correction applies.

Field	Suggested content
task_id	LIBERO task name / later folding task type
state_embedding	Embedding of keyframe or compact multimodal context
failure_signature	Short normalized label + relevant geometric/visual cue
base_action_summary	What the policy attempted to do
correction	Validated intervention type and bounded parameters
outcome	Success/progress delta and whether recovery was sustained
confidence / count	How often this pattern has worked
metadata	Model version, prompt version, timestamps, seed, environment variation

PoC recommendation: we can start with SQLite or JSONL for metadata plus FAISS for nearest-neighbor retrieval over embeddings. This is simpler than deploying a vector database and is enough for a small experiment. If the correction library later grows across many cloth objects, tasks, and robots, we can then migrate the same abstraction to Qdrant or another persistent vector database.

8. Suggested Frameworks and Technologies

Layer	Recommended choice	Why	When to reconsider
Language / orchestration	Python 3.x + dataclasses/Pydantic + asyncio	Lowest integration overhead with LIBERO, PyTorch, and external Python APIs; explicit state machine is easy to debug.	Add LangGraph only if the workflow becomes genuinely multi-agent or branch-heavy.
Simulation	LIBERO / LIBERO-Spatial on robosuite + MuJoCo	Directly matches the requested PoC and provides repeatable manipulation tasks.	Use LIBERO-Pro/perturbations later for robustness testing.
Frozen policy inference	PyTorch + Hugging Face Transformers	Natural fit for a small Hugging Face VLM/policy and supports multimodal inference.	Use vLLM/SQLang only if the chosen model is supported and

			model-serving latency becomes a bottleneck.
Supervisor VLM	Initial small instruction-tuned VLM from Hugging Face; swappable ModelAdapter interface	Lets the team iterate locally and later compare a stronger frontier VLM without changing the harness.	Swap model after the harness works; do not change architecture and model simultaneously.
Structured outputs	Pydantic	Strict validation, bounded fields, clear failure handling.	
Memory	SQLite/JSONL + FAISS	Minimal operations burden; efficient top-k vector search for small correction libraries.	Qdrant when persistence, filtering, concurrent writes, or scale become important.
Logging / artifacts	JSONL + MP4/keyframes + NumPy/Parquet; optional W&B or MLflow	Keeps raw evidence reproducible and enables experiment dashboards without coupling the runtime loop to a platform.	Choose one experiment tracker once the team agrees on deployment constraints.
Configuration	Hydra or simple YAML + dataclasses	Reproducible experiment configs for policy, prompt, seed, task, thresholds, and ablations.	Plain argparse is enough for the very first prototype.
Testing	pytest + deterministic simulator seeds	Unit-test correction bounds, schema validation, and adapter contracts before expensive rollouts.	Add integration CI once environment setup is stable.
Real-robot integration	Neotix-provided Python override API behind RobotAdapter	Keeps project aligned with sponsor infrastructure and avoids premature coupling.	Add ROS 2 only if the sponsor exposes ROS interfaces or it is needed for instrumentation.

8.1 Suggested Repository Structure

```
agentic-harness/  
  configs/  
    libero_poc.yaml  
  harness/  
    orchestrator.py    # state machine / control loop  
    schemas.py         # Pydantic decisions and trace types  
    gating.py          # cheap intervention trigger  
    supervisor.py      # VLM adapter and prompt builder  
    corrections.py     # offset / candidate / recovery logic  
    safety.py          # clipping + validity checks  
    memory.py          # store + retrieve validated corrections  
    evaluator.py       # progress/success verification  
  adapters/  
    libero_env.py      # LIBERO observation/action adapter  
    policy_hf.py       # frozen Hugging Face policy adapter  
    neotix_robot.py    # later: sponsor API adapter  
  logging/  
    trace_logger.py  
  scripts/  
    eval_baseline.py  
    eval_harness.py  
    analyze_results.py  
  tests/  
    test_safety.py  
    test_schema.py  
    test_memory.py
```

9. Latency Strategy

Client's proposal flags VLM latency as a major risk because a call can take a second or more [1]. For the PoC, we can implement latency in three stages:

Stage	Implementation	Goal
A. Synchronous PoC	Pause the decision loop while the VLM decides.	Prove that supervision can improve behavior at all.
B. Gated inference	Call the VLM only on suspicious states and cache repeated diagnoses.	Reduce intervention overhead without changing semantics.
C. Asynchronous pipeline	While chunk n executes, prepare a decision for chunk n+1 or n+2; discard stale decisions if the state changed too much.	Meet real-time constraints for physical deployment.

Measure at least: policy inference time, gate time, VLM time, correction/safety time, simulator step time, total wall-clock per episode, and percentage of VLM decisions that arrive too late. Latency should be reported alongside success rate rather than treated as a hidden engineering detail.

10. Proof-of-Concept Evaluation Plan

Use one LIBERO-Spatial task first. Fix the task, policy checkpoint, seeds, episode limit, and evaluation protocol before adding the harness. Then compare:

- Baseline A - frozen policy only.
- Harness B - frozen policy + gate + supervisor + bounded correction, no cross-episode memory.
- Harness C - same as B + correction memory (after the no-memory version is stable).

10.1 Metrics

Metric	Definition	Why it matters
Episode success rate	Fraction of rollouts completed within the standard horizon.	Primary evidence that the harness helps.
Recovery rate	Fraction of interventions after which the policy returns to a productive trajectory.	Measures intervention quality, not just final success.
Intervention precision	Useful interventions / all interventions.	Detects over-intervention.
Interventions per episode	Mean and distribution.	Lower is preferable at equal success.
Regression rate	Cases where an intervention turns a likely success into failure.	Critical safety/quality metric.
Latency	Mean / p95 supervisor latency and episode wall-clock.	Determines feasibility of online use.
Memory hit benefit	Outcome with retrieved correction vs no-memory matched cases.	Tests whether cross-episode memory contributes.
Failure taxonomy	Counts of misalignment, premature grasp, wrong direction, stalled progress, etc.	Shows where the harness helps and where it still fails.

11. Recommended Implementation Sequence

Phase	Build	Exit criterion	Main deliverable
0. Reproduce baseline	LIBERO environment + frozen policy evaluation	Stable, repeatable baseline across fixed seeds	Baseline report + failure videos
1. Instrumentation	Trace logger, timestamps, keyframe capture, progress evaluator	Every rollout is reconstructable	Trace schema + analysis notebook
2. Harness skeleton	Orchestrator, adapters, Pydantic decision schema, safety shield	Dummy supervisor can PASS/OFFSET end-to-end	Working closed-loop harness
3. Supervisor v1	VLM prompt + gated intervention + bounded OFFSET	At least several manually verified recoveries	PoC demo
4. Controlled evaluation	Baseline vs harness across sufficient seeds	Statistically interpretable success delta	Evaluation report
5. Memory	Validated correction store + FAISS retrieval	Repeated failure patterns reuse prior fixes	Memory ablation
6. Latency optimization	Gating, caching, async inference, batching if applicable	Latency fits target task horizon	Profiling report
7. Physical transfer	Replace LIBERO adapter with Neotix robot adapter; add real safety constraints	Same harness API executes on rig	Towel/T-shirt pilot

12. Relevant Engineering and Research Considerations (AI Suggested)

- **Action semantics:** The supervisor must understand what each action dimension means. Build an action formatter that converts raw vectors/chunks into readable semantics and back.
- **Chunk timing:** If the base policy emits multi-step chunks, decide whether a correction applies to the whole chunk, only the first action, or a regenerated suffix. Log the exact semantics.
- **Stale decisions:** Asynchronous VLM output may refer to an old state. Attach state IDs/timestamps and discard corrections whose observation is no longer current.
- **Over-intervention:** A powerful supervisor can degrade a competent base policy. PASS should be a first-class decision and intervention thresholds should be tuned on validation seeds, not the final test seeds.

- **Memory contamination:** Do not store every intervention. Admit only corrections that are validated by progress or eventual success, with minimum confidence/count thresholds.
- **Prompt leakage / evaluation leakage:** Do not include ground-truth success labels, future observations, or test-specific solutions in the supervisor prompt.
- **Non-determinism:** Run enough seeds and preserve initial states where possible. Report distributions, not only cherry-picked successful videos.
- **Model comparison:** Change one factor at a time. Establish the architecture with one small VLM before comparing stronger VLMs.
- **Transfer to cloth:** LIBERO proves the supervision concept, not cloth manipulation itself. Deformable objects introduce perception and contact dynamics not represented by rigid-object tasks, so transfer should be treated as a separate experimental stage.
- **Physical safety:** On the real rig, hard workspace limits, velocity/acceleration bounds, collision checking, and human e-stop remain outside the LLM and cannot be overridden by it.
- **Reproducibility:** Version model checkpoint, prompt, code commit, config, task seed, dependency lockfile, and memory snapshot for every evaluation run.

13. Minimal Orchestrator Pseudocode

```

obs = env.reset()
memory.reset_episode()

while not done and step < max_steps:
    base = policy.propose(obs, task_instruction)
    risk = gate.score(obs, base, trace.recent())

    if risk < trigger_threshold:
        chosen = base
        decision = PASS
    else:
        examples = memory.retrieve(obs, task_instruction, k=3)
        decision = supervisor.decide(obs, base, trace.summary(), examples)
        chosen = correction_generator.apply(base, decision)

    safe_action = safety.validate_or_fallback(chosen, fallback=base)
    next_obs, reward, done, info = env.step(safe_action)
    trace.log(obs, base, decision, safe_action, reward, info, timings)

    outcome = evaluator.update(obs, next_obs, reward, done, info)
    if outcome.validated_recovery:
        memory.stage_candidate(trace.latest_intervention())

    obs = next_obs

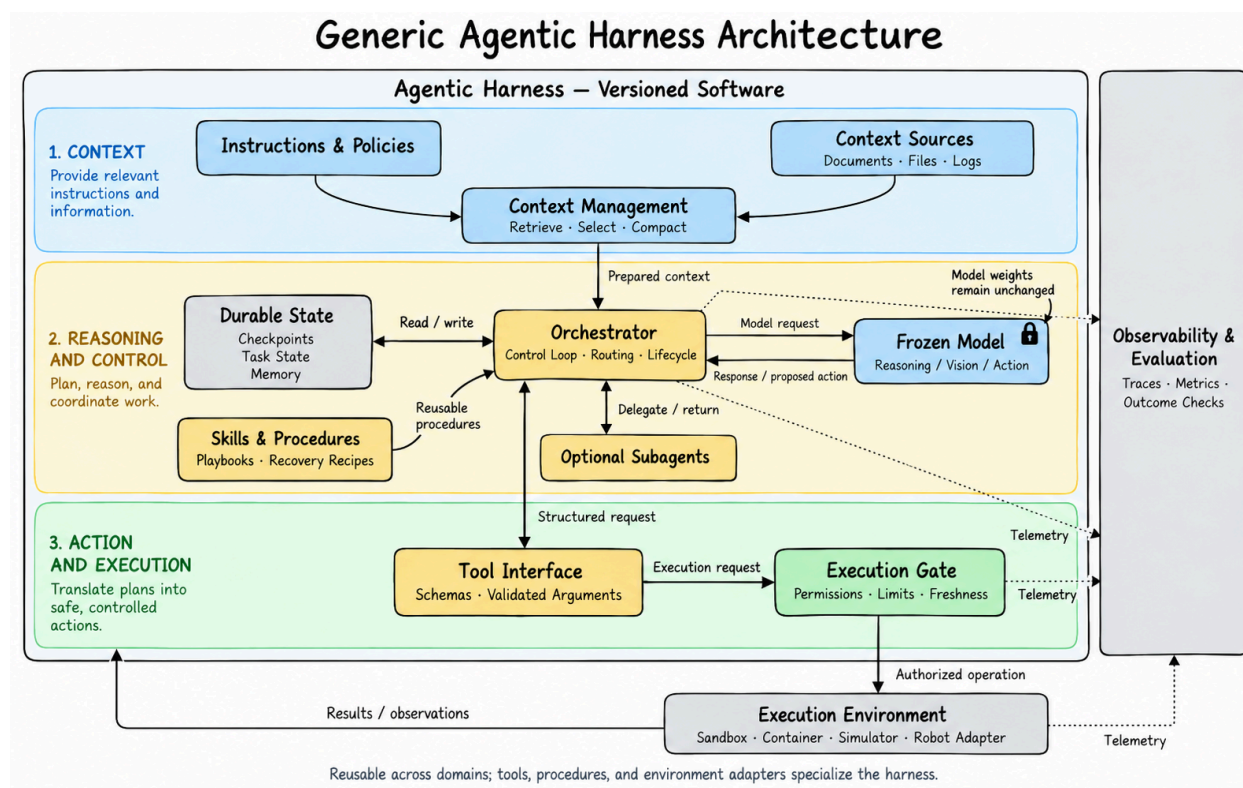
memory.commit_validated(outcome_summary)

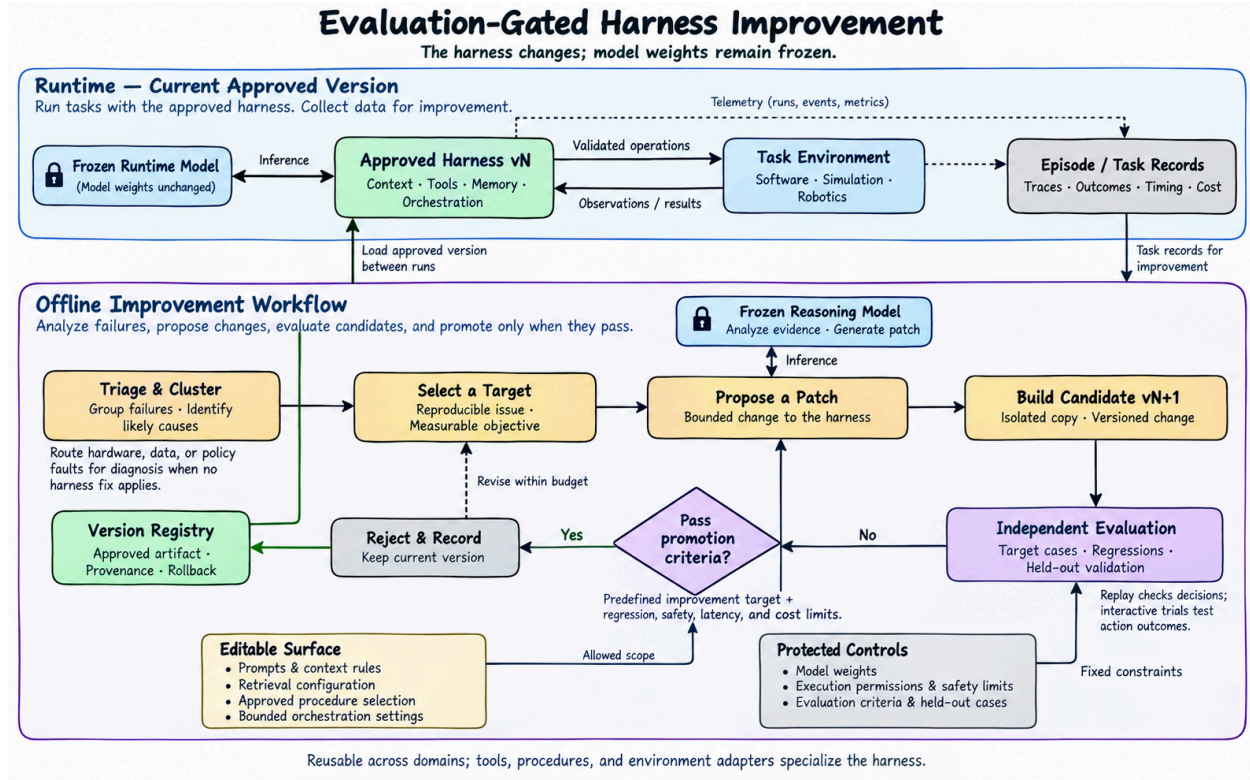
```

14. What Constitutes a Successful PoC?

The first PoC does not need to solve the full latency problem or demonstrate cloth folding. It should establish the core causal claim: on a fixed LIBERO task, the same frozen policy performs better when wrapped by the supervisor, and the improvement can be traced to interpretable interventions rather than extra training. A convincing demo would show at least one recurring failure mode, the supervisor detecting it from deployable observations, a bounded correction that recovers the rollout, and aggregate evaluation showing that the harness improves success without an unacceptable regression rate.

Appendix:





References and Design Sources

- [1] Neotix Robotics Rwanda Ltd. “Agent-Augmented VLA for Bimanual Cloth Manipulation” project proposal. Defines the frozen-policy supervisor loop, action interception options, cross-episode memory, LIBERO simulation requirement, and latency/safety constraints.
- [2] Xiao, W. et al. (2026). ENPIRE: Agentic Robot Policy Self-Improvement in the Real World. Used here for the closed-loop harness framing, automatic verification, safety boundaries, rollout logging, and iterative refinement pattern.
- [3] Lu, R. et al. (2026). ASPIRE: Agentic /Skills Discovery for Robotics. Used here for fine-grained multimodal traces, trace-guided failure attribution, validated repair memory, and reusable skill/correction-library ideas.
- [4] LIBERO: Benchmarking Knowledge Transfer in Lifelong Robot Learning. Official repository: <https://github.com/Lifelong-Robot-Learning/LIBERO>